Chapter 7

HYBRID MODELERS

None of the three major approaches to solid modeling we have described so far is superior to the others in all respects. This motivates the use of multiple simultaneous representations in a serious modeling system. These *hybrid modelers* are the topic of this chapter.

7.1 WHY MULTIPLE REPRESENTATIONS?

In the three previous chapters, the three major approaches to solid modeling were introduced, and their inherent properties discussed. Even at the cost of oversimplification, let us review some key points of this discussion:

1. The simplest of the three approaches, *decomposition models* are superior to the others as sources of data for numerical algorithms, even when they are approximative. Except when the original source of information is a binary image, their creation is complicated without conversion from other representations. They are also voluminous.

2. *Constructive models* are the most concise of the three major approaches to solid modeling. Because the directly available methods for the generation of graphical output or the creation of data for numerical algorithms are slow, these tasks are performed through conversion to other representations.

3. *Boundary models* are directly useful for graphical applications. Especially polyhedral models are useful for numerical applications as well, if the limited modeling space captured by polyhedra can be tolerated. Without the help of other representations and conversions, their creation is difficult, however.

Clearly, a *combination* of the three major approaches to solid modeling would be superior to any of them alone. This has led designers of solid modeling systems to investigate possibilities of combining the basic approaches.

A *hybrid* solid modeler is capable of supporting several coexisting solid representations, and tries to pick the most suitable of them for each task. These representations are guaranteed to be *consistent* up to the level made possible by differences in the inherent modeling spaces. Consistency enforcement is materialized in *conversion algorithms* that can go from one representation to another.

Hence, the representations stored by a hybrid modeler are a combination of several different basic representations. A graphical model can well appear as one of these representations. Observe that a hybrid modeler can well include, say, several types of boundary models. For instance, it may store a winged-edge representation with curved surfaces for modeling procedures, and a distinct polyhedral model for display purposes. Solid representations for secondary storage could also form still another distinct boundary model.

7.2 PROBLEMS OF HYBRID MODELERS

The coexistence of several solid representations in one modeler raises a number of new kinds of problems.

7.2.1 Conversions

Clearly, the conversion algorithms form a fundamental part of a hybrid modeler. Unfortunately, there are inherent limitations on what kinds of conversions a hybrid modeler can expect to support.

Constructive models and CSG models in particular have the virtue that they can be converted to any other representation. The feasibility of the conversion to a boundary model, the *boundary evaluation* has been well demonstrated in many research and commercial CSG modelers. General approaches for converting from a CSG representation to an enumerative representation were outlined in Chapter 5.

Unfortunately, the inverse conversions are not available. In particular, the *inverse boundary evaluation*, i.e., the conversion from a boundary representation to a CSG model, still remains an unsolved problem for practical purposes in the general 3-dimensional case. However, boundary models can be converted to decomposition models approximately as easily as CSG.

The conversions from a decomposition model to a boundary model or a CSG model would be valuable in the case that information received as a binary image must be mapped against information stored as a boundary model or a CSG model. Robot vision offers an example of great practical interest. Unfortunately, these conversions are still more a research subject than a technology that could be exploited in practical solid modelers.

Because conversions from boundary modes or CSG models to decomposition models generally are approximative, it may seem that the inverse conversion makes little sense: if the exact information is already available, why attempt its reconstruction? Solid modelers based on boundary models or CSG models usually assume this view, and treat decomposition models as *transient* representations, i.e., decomposition models are created "on the fly" to solve certain problems, and discarded afterwards.

In fact, *all* modelers that create graphical output during an interactive design session can be considered hybrid modelers, because the display is a kind of geometric model of its own.

7.2.2 Consistency

If a conversion algorithm between two representations included in a hybrid modeler is available, it will be possible to keep the representations consistent. Unfortunately, this usually happens at the cost of limiting the functionality available in the modeler.

For instance, CSG models cannot ordinarily include parametric surfaces. This means that if a boundary model created by conversion from CSG is modified further to include blends represented as parametric surfaces, the original CSG representation of the object modeled cannot be kept consistent with it.

In general, a hybrid modeler that aims at total consistency between the representations can only support such operations on solids that can be mapped to each of the representations. Boolean set operations have their prominent position in solid modeling partly because they can be mapped to all the major solid representations. Unfortunately, set operations alone are not sufficient for constructing a good user interface to a solid modeler, and it is much more difficult to support "drawing" operations on several representations.

7.2.3 Modeling Transactions

All modifications to solid representations take place in a sequential fashion. Therefore, a total consistency between the various representations cannot exist at all time instances.

This leads us to define the concept of a *modeling transaction* as an indivisible sequence of modeling operations that commences from a consis-

tent state and ends in a consistent state. For instance, in a hybrid modeler consisting of a CSG modeler and a polyhedral modeler, the modeling transaction of performing a set operation on two primitives would consist of the creation of a CSG tree consisting of the primitives, and computing a polyhedral model of the result of the set operation. In an interactive environment, the display can be considered a representation in its own right; in this case, the modeling transaction should include the updating of the displayed image. Solid representations in secondary storage add another dimension to modeling transactions: an interactive modeling session is complete only after the relevant results are stored.

The functions available to the user must be designed with care in any solid modeler. In a hybrid modeler the careful design becomes still more important.

7.3 HYBRID ARCHITECTURES

The solution usually adopted in hybrid modelers to the problems of lacking conversions and consistency is to treat one of the representations supported as the *primary* representation, and the others as *secondary* representations.

In this respect, most solid modelers available today seem to fall into one of two major architectures. The modelers in the first category (Figure 7.1(a)) ordinarily use CSG trees as their primary solid representations. From CSG trees, boundary models may be created through boundary evaluation, and decomposition models through classification algorithms in the style discussed in Section 5.2.

The user, however, has no direct access to the secondary representations. In other words, although boundary models might be included in the modeler (and perhaps continuously updated through incremental boundary evaluation to reflect changes in the CSG tree), the user of the modeler cannot perform modeling operations which are specific to a boundary model, say, local modifications. Modelers such as PADL-1 [127], PADL-2 [21], and GMSOLID [17] belong to this group.

Modelers that use boundary models as their primary representation form the second group (Figure 7.1(b)). These modelers usually include CSG as one solid description facility, and many of them also store a variation of the CSG tree (see e.g. [18]). In addition to CSG, however, these modelers have also other solid description facilities that directly modify boundary data structures. These facilities may include the "drawing" and "sweeping" operations or local solid modifications as discussed in Chapter 6. Hence, the role of CSG in these systems is really that of an auxiliary representation only. Many industrially used modelers, such as ROMULUS,

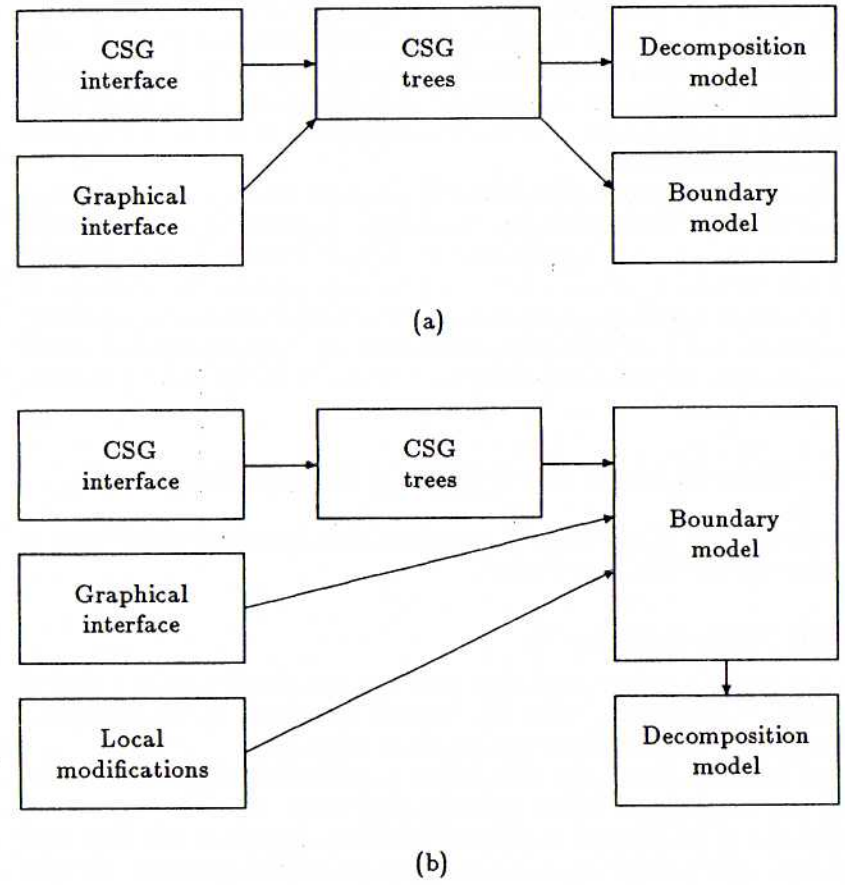

Figure 7.1 Architectures for hybrid modelers.

GEOMOD [118], and MEDUSA [87] belong to the second major group.

Various architectures for hybrid modelers are discussed in greater detail in references [99,100], including information of numerous solid modelers.

7.4 DISTRIBUTED MODELERS

The current trend towards the use of *distributed* graphics systems consisting of a host computer and graphics workstations has significance to hybrid solid modelers. As current graphics workstations have considerable processing power of their own, and actually are usually small computers themselves, the problem of dividing the labor sensibly between the host and the workstations becomes urgent. The analogous problem also arises in top-of-the-line workstations that have a special graphics processor for the rapid display of polygonal models.

A solution to this dilemma is the introduction of a *distributed modeler*, part of which resides in a workstation and part in the host. Because the modeler maintains solid models in both parts, distributed modelers are a special case of hybrid modelers.

As an example of the division of labor in a distributed modeler in the context of CSG models, Atherton [6] suggests the separation of a *visual modeler* from a *analytical modeler*. The visual model is intended to support rapid interactive processing, and resides locally in the workstation. The analytical model is stored in the (still quintessential) host computer; for CSG, the analytical model is the exact CSG tree. In particular, Atherton suggests that CSG primitives are approximated by polyhedra in the workstation, and proposes an efficient algorithm for rapidly displaying CSG trees consisting of such polyhedral primitives.

A similar division of labor is also possible in boundary models. So-called *faceting modelers* are a class of boundary modelers that "know" about curved surfaces, but do not represent them explicitly. Rather, they store polyhedral approximation of the surface, and it is this representation that is used for, say, generation of graphical output. The distinction between faceting modelers and what we have called merely polyhedral modelers is that polyhedral modelers do not know where the planar faces came from, even if they can create polyhedral approximations of curved objects.

Many graphics workstations store a structured display list with polygonal graphical primitives, which can be considered a simple polyhedral model. The display list is constructed by a boundary modeler residing in the host computer, and it must be updated to reflect operations performed on the solid model. Unfortunately, this architecture still poses heavy requirements for host-workstation communication in the distributed

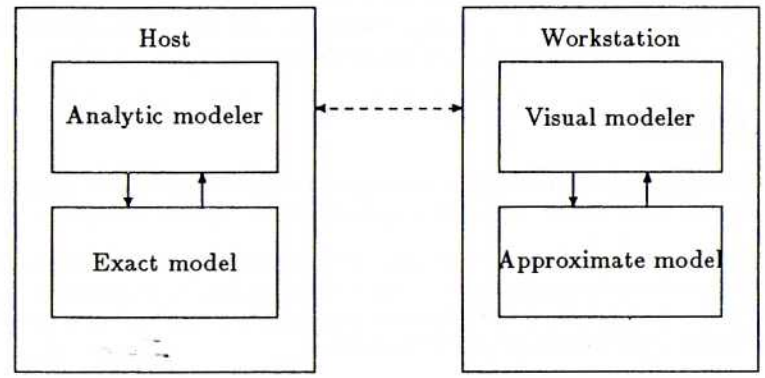

Figure 7.2 A distributed modeler.

environment.

As a step forward, a faceting modeler could be used in the role of a visual modeler, i.e., to provide rapid interactive processing locally in a workstation. This approach is strongly supported by that many graphics workstations have special hardware for rapid processing of polyhedral models. For numerical applications requiring high accuracy, the implicit surface information of the faceted modeler can be transmitted to the host computer, and elaborated by the analytic modeler.

7.5 OTHER HYBRID APPROACHES

Hybrid modelers discussed in the previous sections use several coexisting independent representations to be able to use the most appropriate ones for each task.

To avoid the inherent difficulties caused by several represenrations, modelers that pursue the combination of several basic approaches in a single integrated solid representation scheme have been proposed.

Typically, these modelers combine a decomposition model (exhaustive enumeration or an octree) with either a CSG model or a boundary model, and use the decomposition model as a geometric access method to CSG or boundary information.

For instance, the *extended octree* of Ayala *et al.* [7,86] is based on storing a boundary representation of a polyhedral object within the cells of an octree. The octant subdivision is continued until each cell contains at

most one vertex, one edge, one face, or is homogeneously "full" or "empty." The cited references give algorithms for conversion from a boundary representation to an extended octree and vice versa, and for set operations of extended octrees. References [22,136] describe similar representations.

PROBLEMS

7.1. Are there other solid description techniques besides set operations that can easily be supported on top of all major approaches to solid modeling?

7.2. The separation between "general-purpose" geometric models and graphical data structures (e.g., structured display lists) for display purposes is debatable. Consider the pros and cons of keeping these data separate versus integrating them into a single representation.

BIBLIOGRAPHIC NOTES

An algorithm for inverse boundary evaluation is expected to convert an arbitrary boundary model into an equivalent CSG tree. Unfortunately, the problem seems to be subject to combinatorial explosion, and rather difficult to solve optimally (i.e., with the minimum number of primitives) or even satisfactorily.

The feature extraction algorithm of Woo [130] can *in principle* generate a CSG expression from a boundary model by using a convex hull technique. Unfortunately, without human help the algorithm seems to be subject to infinite looping.

Two-dimensional versions of the problem are much simpler. See, for instance, the work of Vossler [128] on the practically interesting problem of the conversion of swept planar outlines consisting of straight lines and circular arcs to a CSG model.

As for the other conversions, Kunii *et al.* [66] have reported an algorithm for converting an octree to a boundary representation. However, the algorithm makes no attempt to infer the actual surfaces of a curved object from the rasterized data.